# Game Design Document: The Slime's Warden v3.0

**Género:** Plataformas 2D de escalada vertical
**Motor:** Unity 2022.3 LTS

## 1. Historia y Premisa Narrativa

---

### La Misión

El jugador controla a un "Brujo" que debe escalar un inmenso abismo. El objetivo principal no es solo subir, sino actuar como el guardián heroico (Warden) de un pequeño y frágil mini slime llamado **Squishy**, escoltándolo a salvo hasta la cima para que se reúna con su familia.

### Sistema de Diálogos y Eventos Narrativos

La narrativa se apoya en un sistema de diálogos gestionado por *ScriptableObjects* (contenedores de datos) e interfaces de usuario dinámicas (*DialogueUI*). Esto permite eventos interactivos como donde Squishy y el Brujo tienen un encuentro directo:

- **Intro:** El slime pide ayuda activamente a través de diálogos emotivos ("*¡Ay! ¡Hola, señor Brujo! Qué suerte que te encuentro acá... Me perdí... Mi familia está re lejos allá arriba y me da mucho miedito ir solo.*"), cimentando el vínculo inicial y estableciendo la meta del juego.
- **Triggers Narrativos:** Existen zonas en los niveles (como *LevelIntroDialogue* y *OutroDialogueTrigger*) que activan interacciones específicas para avanzar la historia sin detener de forma intrusiva la experiencia de plataformeo.

## 2. Mecánicas de Juego

---

### Personaje y Movimiento (Character)

El movimiento busca ser ágil y extremadamente preciso, diseñado para que los errores sean culpa del jugador y no del sistema.

- **Físicas:** Movimiento horizontal manipulando directamente la *velocity* del Rigidbody2D, preservando la inercia vertical.
- **Detección de Suelo:** Optimizada mediante un *Physics2D.BoxCast*, evitando falsos negativos en

bordes de plataformas.

- **Colisiones:** Uso de un CapsuleCollider2D con material físico de fricción cero para evitar atascos en las costuras de los tiles.
- **Animación:** Máquina de estados en el Animator sin tiempos de transición (blending) para un "Game Feel" ágil e instantáneo.

### Cámara (Camera)

El seguimiento es estrictamente vertical, gestionado por una cámara virtual de Cinemachine.

- **Dead Zone:** Zona muerta horizontal configurada al máximo para permitir movimiento libre de izquierda a derecha sin paneos innecesarios.
- **Framing Offset:** Mantiene al jugador en el tercio inferior de la pantalla, otorgando tiempo de reacción para ver peligros u obstáculos superiores.
- **Confinamiento:** CinemachineConfiner2D asociado a un colisionador poligonal asegura que la lente nunca muestre el fondo vacío fuera del mapa jugable.

### Controles (Controls)

Implementados a través del "New Input System" de Unity. Se usa un patrón "PlayerBlackboard" para desacoplar la lectura de inputs de las físicas reales.

| Acción | Input Principal | Lógica de Sistema |
|---|---|---|
| Moverse | A / D | Polling continuo |
| Saltar | Space | Por eventos (milisegundos) |
| Disparar | Enter | Ejecución con Cooldown |

## 3. Combate, Supervivencia y Optimización

---

### Sistema de Combate Táctico

El juego introduce una capa de gestión de recursos. Los proyectiles mágicos son finitos y el jugador debe pensar bien cada disparo.

- **Munición y Cooldown:** Un temporizador previene el "spam" de ataques. Para recargar, el jugador

debe recolectar cajas de munición (AmmoPickups) dispersas en el mapa.

- **Game Feel (Audio/VFX):** Cada proyectil reproduce sonidos direccionales de lanzamiento e impacto, sumado a partículas de explosión en la colisión para otorgar peso visual y sonoro a la acción.
- **Optimización Estricta (Object Pooling):** Para mantener un rendimiento fluido libre de tirones, todas las balas, partículas de impacto y reproductores de sonido se gestionan usando la API nativa UnityEngine.Pool.

# 4. Inteligencia Artificial (Habitantes del Mundo)

---

| Entidad | Comportamiento e IA |
|---|---|
| **Squishy (Mini Slime)** | ● Persigue la coordenada X del jugador suavizando el movimiento mediante *Mathf.SmoothDamp*.<br>● Detecta el terreno por adelantado usando un "Dron/Radar" invisible que dispara un *Physics2D.Raycast* vertical hacia su futura posición.<br>● Ejecuta un "Salto de Fe" mágico para flotar junto al jugador al detectar un pozo profundo.<br>● Cuenta con un Offset matemático en el eje Y para alinear permanentemente su flotación a los pies del jugador. |
| **Slimes Patrulleros** | ● Enemigos hostiles que patrullan de lado a lado (estilo Goomba).<br>● Utilizan dos sensores de *Physics2D.Raycast* (uno frontal y otro inferior) para detectar paredes o bordes de precipicios y darse vuelta automáticamente. |
| **Slimes Francotiradores** | ● Enemigos estacionarios (carecen de rutina de patrullaje) pero que poseen un **gran alcance visual y de ataque**.<br>● Actúan como torretas, disparando proyectiles hacia el jugador, lo que obliga al Brujo a tomar un enfoque más táctico (evadir, bloquear con el entorno o contraatacar) durante el ascenso. |

# 5. Diseño de Nivel y Entorno

El mundo está diseñado para ser progresivamente hostil, apoyado en bases técnicas limpias.

- **Construcción Estructural:** El mapa se elabora mediante *Tilemaps* unificados con *CompositeCollider2D* para crear una geometría fluida y lisa, evitando interrupciones en el movimiento.
- **Plataformas Frágiles (Estrés Rítmico):** Elementos clave de *Level Design* que detectan el peso del jugador, inician una animación de temblor, y tras 1 a 2 segundos desactivan su colisión cayendo al vacío, lo que fuerza al jugador a mantener un ritmo de ascenso.
- **Trampolines (Dinámica de Salto):** Superficies interactivas dispuestas estratégicamente en el nivel que impulsan al jugador a grandes alturas al saltar sobre ellas. Estas mecánicas facilitan la exploración vertical, aportan variedad rítmica y saltos cinéticos.